

Cocotb Report

Marium Waleed Mostafa

July 2026

Design



```
1  module alu #(parameter n = 4) (  
2      input wire      CLK,  
3      input wire      RST,  
4      input wire [n-1:0] A,  
5      input wire [n-1:0] B,  
6      input wire [1:0] OpSel,  
7      output reg [n-1:0] Out,  
8      output reg      Carry  
9  );  
10  
11      localparam OP_ADD = 2'b00;  
12      localparam OP_SUB = 2'b01;  
13      localparam OP_AND = 2'b10;  
14      localparam OP_OR  = 2'b11;  
15  
16      always @(posedge CLK or negedge RST) begin  
17  
18          if(!RST) begin  
19              Out = 4'h0;  
20              Carry = 1'b0;  
21          end else begin  
22              case (OpSel)  
23                  OP_ADD: {Carry, Out} = A + B;  
24                  OP_SUB: {Carry, Out} = A - B;  
25                  OP_AND: {Carry, Out} = A & B;  
26                  OP_OR:  {Carry, Out} = A | B;  
27                  default: {Carry, Out} = 5'h0;  
28              endcase  
29          end  
30  
31      end  
32  
33  endmodule
```

Figure 1: Design Code

Testbench



```
1 import random
2
3 import cocotb
4 from cocotb.clock import Clock
5 from cocotb.triggers import *
6 from cocotb.handle import Force, Release, Freeze, Deposit
7
8 OP_ADD = 0b00
9 OP_SUB = 0b01
10 OP_AND = 0b10
11 OP_OR  = 0b11
12
13 N      = 4
14 MASK   = (1 << N) - 1
15 EXT_MASK = (1 << (N + 1)) - 1
16
17
18 def expected_result(a, b, opsel):
19     """Golden model matching the RTL's {Carry, Out} <= ... behaviour."""
20     if opsel == OP_ADD:
21         res = (a + b) & EXT_MASK
22     elif opsel == OP_SUB:
23         res = (a - b) & EXT_MASK
24     elif opsel == OP_AND:
25         res = (a & b) & EXT_MASK
26     else:
27         res = (a | b) & EXT_MASK
28
29     carry = (res >> N) & 1
30     out   = res & MASK
31     return out, carry
```

Figure 2: Testbench Code (1)

```

1  async def reset_dut(dut, cycles=3):
2      dut.RST.value = 0
3      dut.A.value   = 0
4      dut.B.value   = 0
5      dut.OpSel.value = 0
6
7      for _ in range(cycles):
8          await FallingEdge(dut.CLK)
9
10     dut.RST.value = 1
11     await FallingEdge(dut.CLK)
12     dut._log.info("Reset released")
13
14
15  async def drive_and_check(dut, a, b, opsel, op_name):
16      await FallingEdge(dut.CLK)
17      dut.A.value = a
18      dut.B.value = b
19      dut.OpSel.value = opsel
20
21      await RisingEdge(dut.CLK)
22      await ReadOnly()
23
24      exp_out, exp_carry = expected_result(a, b, opsel)
25      got_out = int(dut.Out.value)
26      got_carry = int(dut.Carry.value)
27
28      if got_out == exp_out and got_carry == exp_carry:
29          dut._log.info(
30              f"PASS [{op_name}] A={a} B={b} -> Out={got_out} Carry={got_carry}"
31          )
32          return True
33
34      dut._log.error(
35          f"FAIL [{op_name}] A={a} B={b} OpSel={opsel:02b} -> "
36          f"got Out={got_out} Carry={got_carry}, "
37          f"exp Out={exp_out} Carry={exp_carry}"
38      )
39      return False

```

Figure 3: Testbench Code (2)

```

1  async def arithmetic_driver(dut, n_transactions=25):
2      passed = 0
3      for _ in range(n_transactions):
4          a      = random.randint(0, MASK)
5          b      = random.randint(0, MASK)
6          opsel  = random.choice([OP_ADD, OP_SUB])
7          op_name = "ADD" if opsel == OP_ADD else "SUB"
8          passed += await drive_and_check(dut, a, b, opsel, op_name)
9
10     dut._log.info(f"Arithmetic driver done: {passed}/{n_transactions} passed")
11     return passed, n_transactions
12
13
14  async def logic_driver(dut, n_transactions=25):
15      passed = 0
16      for _ in range(n_transactions):
17          a      = random.randint(0, MASK)
18          b      = random.randint(0, MASK)
19          opsel  = random.choice([OP_AND, OP_OR])
20          op_name = "AND" if opsel == OP_AND else "OR"
21          passed += await drive_and_check(dut, a, b, opsel, op_name)
22
23     dut._log.info(f"Logic driver done: {passed}/{n_transactions} passed")
24     return passed, n_transactions
25
26
27  @cocotb.test()
28  async def tb_top(dut):
29      dut._log.info("STARTING SIMULATION")
30
31      clk = Clock(dut.CLK, 10, units="ns")
32      await cocotb.start(clk.start())
33
34      await reset_dut(dut)
35
36      dut._log.info("=== Running arithmetic driver (ADD/SUB) ===")
37      arith_pass, arith_total = await arithmetic_driver(dut, n_transactions=25)
38
39      dut._log.info("=== Running logic driver (AND/OR) ===")
40      logic_pass, logic_total = await logic_driver(dut, n_transactions=25)
41
42      total_pass = arith_pass + logic_pass
43      total_txn  = arith_total + logic_total
44      dut._log.info(f"SUMMARY: {total_pass}/{total_txn} transactions passed")
45
46      assert total_pass == total_txn, f"{total_txn - total_pass} transaction(s) failed!"
47

```

Figure 4: Testbench Code (3)

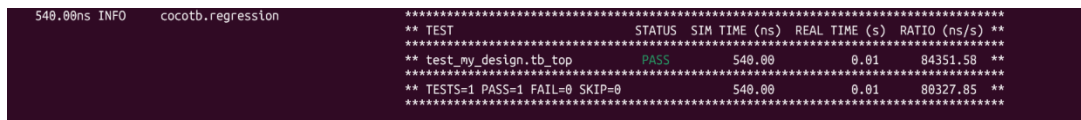
Makefile

A terminal window with a dark background and three colored window control buttons (red, yellow, green) in the top left corner. The terminal displays a Makefile script with the following content:

```
1  SIM ?= icarus
2  TOPLEVEL_LANG ?= verilog
3
4  VERILOG_SOURCES += $(PWD)/my_design.v
5  TOPLEVEL = alu
6  MODULE = test_my_design
7
8  include $(shell cocotb-config --makefiles)/Makefile.sim
9
```

Figure 5: Makefile

Simulation

A terminal window with a dark purple background showing the output of a simulation. The output includes a header line, a table of test results, and a summary line. The table has columns for TEST, STATUS, SIM TIME (ns), REAL TIME (s), and RATIO (ns/s). The test 'test_my_design.tb_top' is shown with a status of 'PASS'.

```
540.00ns INFO cocotb.regression
*****
** TEST                STATUS SIM TIME (ns) REAL TIME (s) RATIO (ns/s) **
*****
** test_my_design.tb_top  PASS      540.00      0.01    84351.58 **
*****
** TESTS=1 PASS=1 FAIL=0 SKIP=0      540.00      0.01    80327.85 **
*****
```

Figure 6: Simulation Results